Embedded Software and Deterministic Multitasking

Scheduling Architecture and UART (Universal Asynchronous Receiver-Transmitter)

Session 02



Contents

01	**Blocking and Non-Blocking Execution**	Blocking IO problems and its solution
02	**Multitasking and Scheduling**	The architecture of Task Scheduling
03	**UART**	Serial Communication
04	**Implementation**	Connecting the Theory to Physical System

The Problem with Blocking Waits

```
void blocking_wait(uint32_t ms) {
  uint32_t start = millis();
  while (uint32_t(millis() - start) < ms) {
    // No other task in loop() can run here.
  }
}
```

Conceptual busy wait: what happens to other loop tasks?

Source: course PDF, pp. 1–2

What Actually Stops?

Busy wait	Keeps the calling task occupied until the time interval expires.
ESP32 delay()	Yields the calling FreeRTOS task; other RTOS tasks can still run.
Our loop tasks	Cannot run until loop() regains control. Hardware PWM continues independently.

Blocking loop() delays the cooperative schedule.

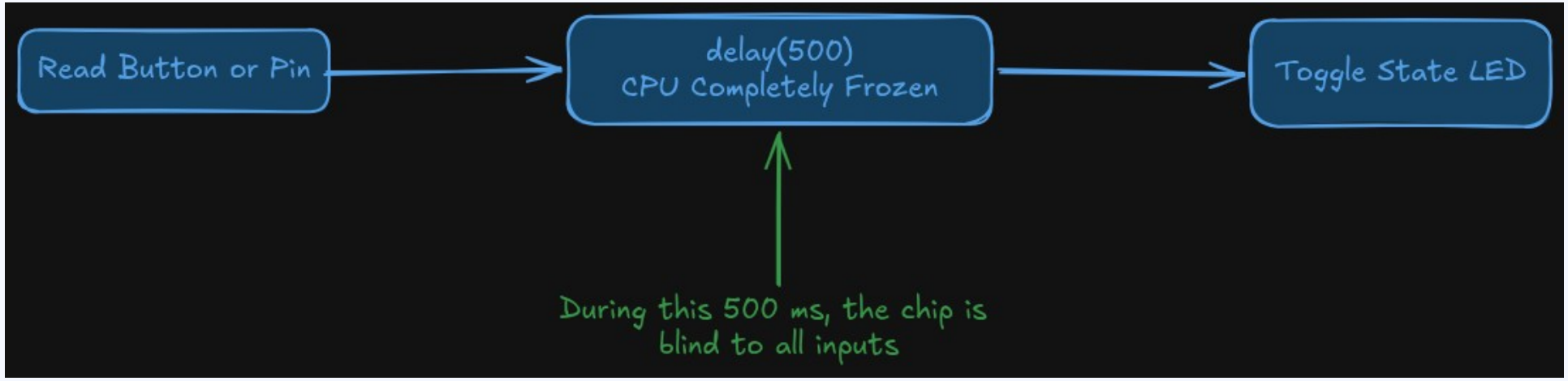
Source: course PDF, pp. 1–2

The Problem with Delay

During this pause:

Missed input polling
Unread serial data can overflow
Delayed software output updates

So, how do we solve this?



Timestamp Delta Subtraction

The provided functions

- millis(): returns the number of milliseconds elapsed since the chip booted as a 32-bit unsigned integer (uint32_t)
- micros(): returns the number of microseconds elapsed since the chip booted as a 32-bit unsigned integer (uint32_t)

```
I (820) example: Periodic timer called, time since boot: 532169 us
I (1320) example: Periodic timer called, time since boot: 1032139 us
I (1820) example: Periodic timer called, time since boot: 1532139 us
I (2320) example: Periodic timer called, time since boot: 2032139 us
periodic           500000      2532116         1         4      0          726
one-shot                0      5032128         1         0      0            0
I (2820) example: Periodic timer called, time since boot: 2532139 us
I (3320) example: Periodic timer called, time since boot: 3032139 us
I (3820) example: Periodic timer called, time since boot: 3532139 us
I (4320) example: Periodic timer called, time since boot: 4032139 us
periodic           500000      4532116         1         8      0         1406
one-shot                0      5032128         1         0      0            0
I (4820) example: Periodic timer called, time since boot: 4532139 us
I (5320) example: Periodic timer called, time since boot: 5032135 us
```

What they do

$$\Delta t = t_{current} - t_{previous}, \Delta t \geqslant T_{interval}$$

$$if\ \Delta t < T_{internal}, the\ CPU\ skips\ the\ task$$

$$if\ \Delta t > T_{internal}, the\ CPU\ executes\ the\ task$$

Periodic Scheduling: Relative Reset

```
if (uint32_t(now - previous) >= INTERVAL) {
  previous = now;
  execute_task();
}
```

A late execution shifts the next scheduled time.

Source: course PDF, pp. 3

>_

This code causes cumulative timebase drift, why?

If a 10 ms task runs at 12 ms, resetting the timestamp to 12 ms schedules the next run for 22 ms. The 2 ms delay shifts the future schedule.

Periodic Scheduling: Absolute Anchoring

```
if (uint32_t(now - previous) >= INTERVAL) {
  previous += INTERVAL;
  execute_task();
}
```

Advance by the period to keep the original time grid.

Source: course PDF, pp. 3

>_

This code is the standard method for periodic scheduling

Advance the timestamp by the fixed task period. A late run at 12 ms still leaves the next target at 20 ms. This prevents schedule drift, but does not remove lateness.

When Do 32-Bit Timers Wrap?

`micros()`

71.58 minutes

2^32 microseconds
Then the counter
returns to zero.

`millis()`

49.71 days

2^32 milliseconds
Use unsigned elapsed
time subtraction.

A wrapping timestamp is normal; compare elapsed durations.

Source: course PDF, pp. 3–5

Forward Addition

```
uint32_t deadline = millis() + INTERVAL;

void loop() {
    if (millis() >= deadline) {
        deadline = millis() + INTERVAL;
        execute_task();
    }
}
```

What does this code do? Would it fail?

>_

This code would fail

millis() wraps from 4,294,967,295 back to 0 while deadline is an absolute timestamp, the >= comparison stops reflecting real time order near the rollover, so the task misfires (or stalls) until the two values line up again.

Unsigned Subtraction Handles Rollover

```
if (uint32_t(now - previous) >= INTERVAL) {
  previous += INTERVAL;
  execute_task();
}
```

Example: uint32_t(5 - 4294967290) gives 11 elapsed ticks.

Source: course PDF, pp. 4–5

>_

This code would work

Unsigned subtraction wraps modulo 2^{32}. For example, 5 – 4,294,967,290 gives 11 elapsed ticks. Check often enough that less than one full timer wrap passes.

Execution Budgets and Timing

PERIOD

10,000 us

100 Hz means one release every 10 ms.

CPU LOAD

T_exec / T_period

Example: 20 us / 10,000 us = 0.2%.

LATENESS

Actual – target

How late the task starts relative to its scheduled slot.

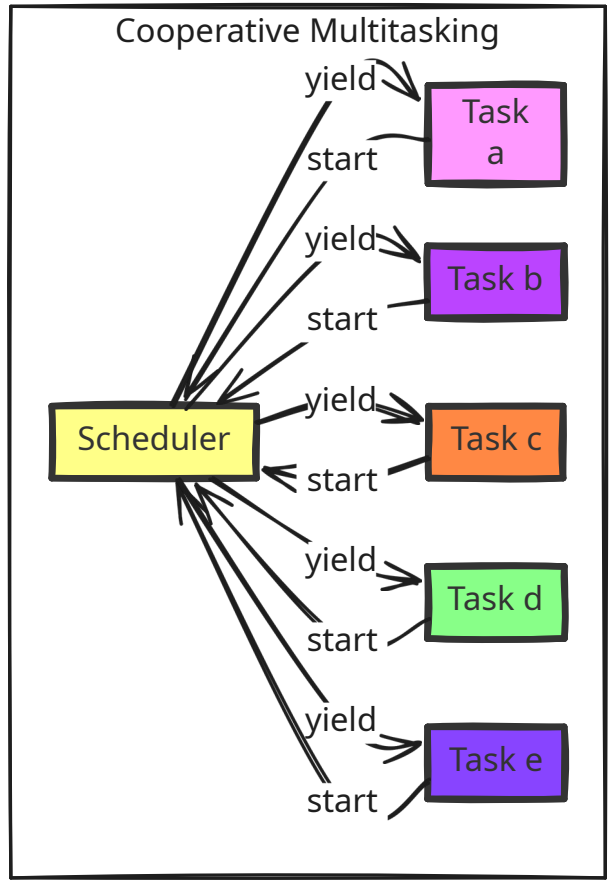
Keep tasks short and leave time for other work.

Source: course PDF, pp. 5–6

Principles of Cooperative Multitasking

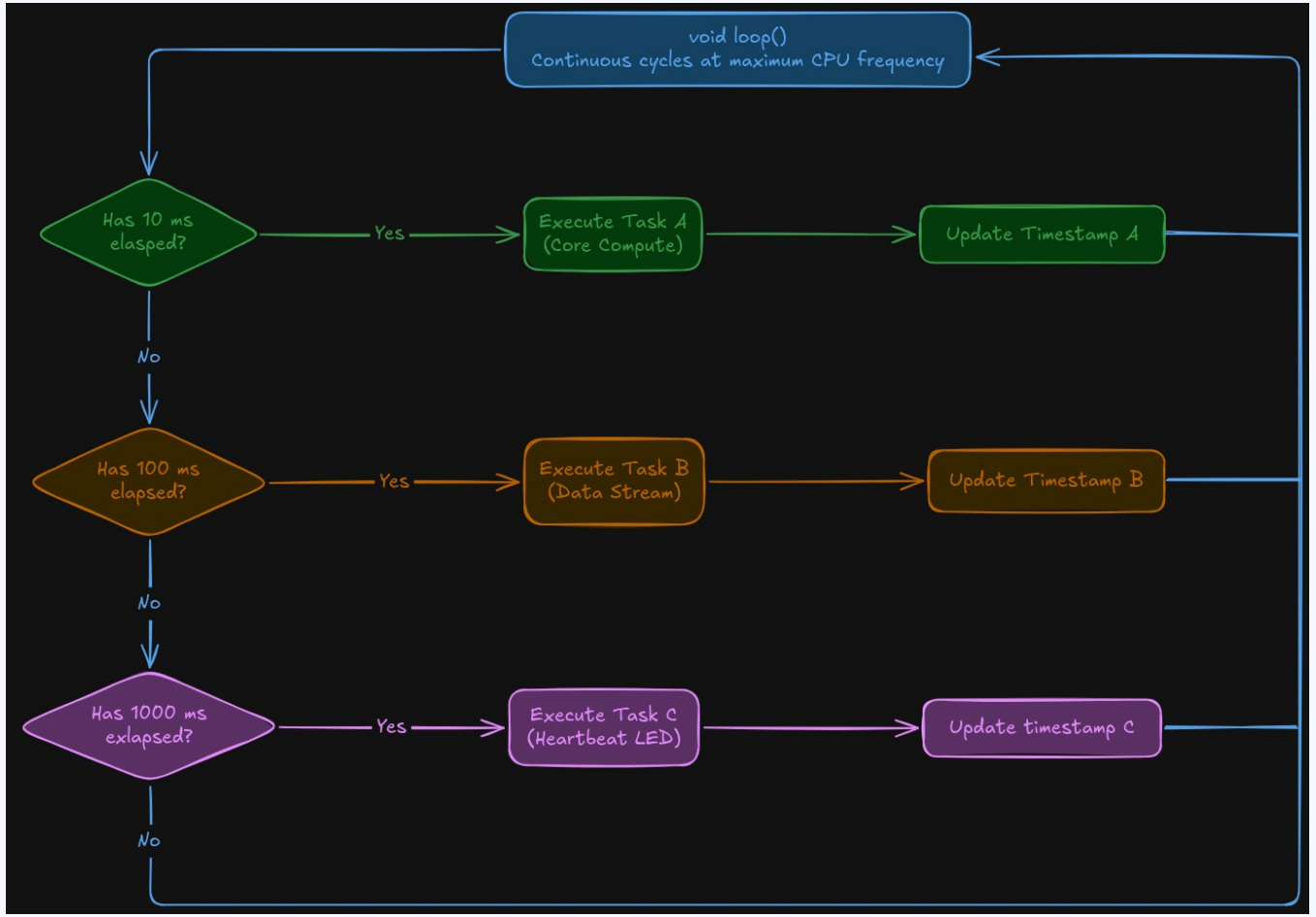
Cooperative (Non-Preemptive) Multitasking

These functions run cooperatively inside loop(). ESP32 still runs FreeRTOS underneath.
Each function does a small amount of work and returns promptly.
A long task delays the others in this loop.



Cooperative Non-Blocking Task Cycle

- Every task function must execute and return in a fraction of its total period
- Never call delay() or long blocking while() loops inside periodic task



A Reusable Periodic Task

```
struct PeriodicTask {
  uint32_t last = 0, period;
  bool ready(uint32_t now) {
    if (uint32_t(now - last) < period) return false;
    last += period;
    return true;
  }
};
```

Create independent timers, for example: PeriodicTask motion{0, 10000};

Source: course PDF, pp. 7–8

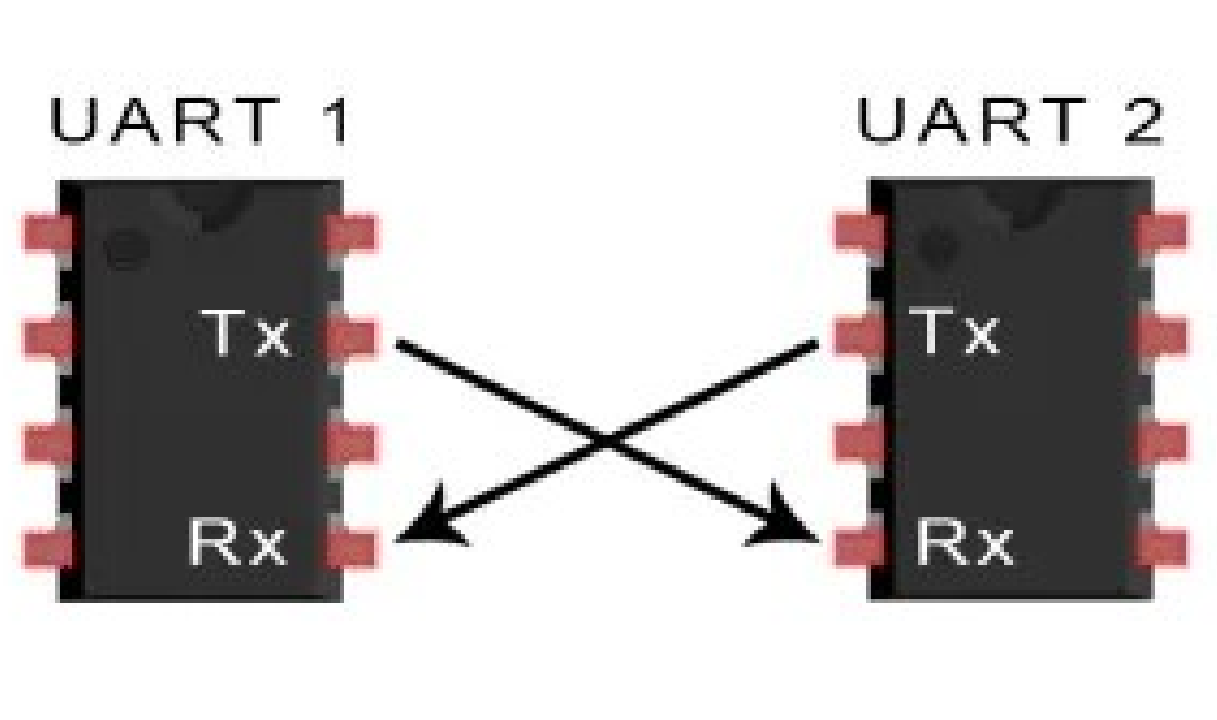
>_

This is how we structure Multi-Rate Periodic Tasks

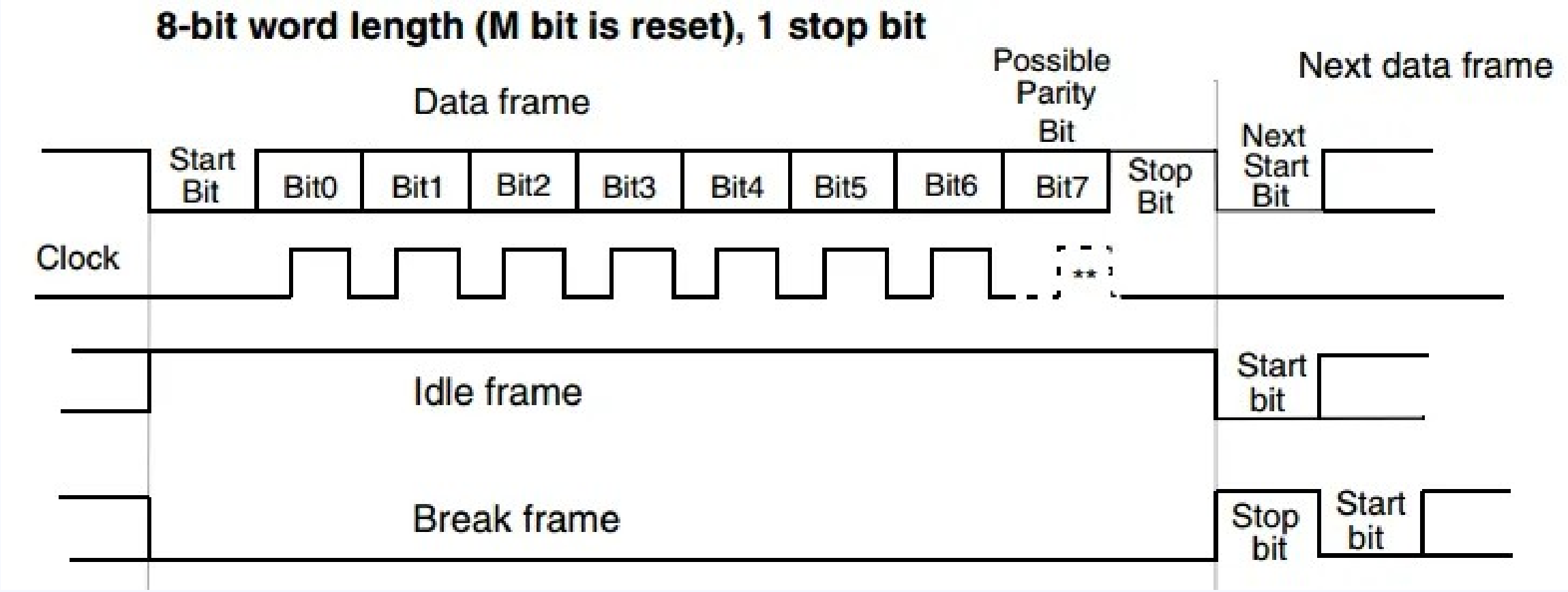
Run fast computational tasks at 100 Hz and slower I/O tasks at 10 Hz to maintain responsiveness and avoid overloading communication buses.

UART Physical Sampling and 8-N-1 Asynchronous Framing

UART has no shared clock; both ends must agree on baud rate and framing.
Mark (logic 1): idle HIGH, about 3.3 V on this ESP32.
Space (logic 0): LOW, near GND.



UART Frame Structure



UART Frame: What Each Bit Does

Start: 1 bit	The line changes from idle HIGH to LOW to mark the beginning of the frame.
Data: 8 bits	Transmit the payload one bit at a time, least significant bit first.
Stop: 1 bit	Return to HIGH for at least one bit time before the next frame.

8-N-1: 1 start + 8 data + 0 parity + 1 stop = 10 bits per byte.

Source: course PDF, pp. 9

UART Capacity at 115200 Baud

ONE BIT	ONE BYTE	MAXIMUM RATE
8.68 us	**86.81 us**	**11,520 B/s**
1 / 115200 seconds	8-N-1 uses 10 bits: 1 start + 8 data + 1 stop.	115200 / 10 Payload bytes per second.

The UART framing overhead counts toward the wire bandwidth.

Source: course PDF, pp. 9

Too Much Logging in a Fast Task

```
void task_100Hz() {
  // Assume each formatted message is 120 bytes.
  Serial.printf("Cycle: %lu, Value: %.2f ...\n",
                cycle_count, value);
}
```

120 bytes × 100 Hz = 12,000 B/s: more than UART can transmit.

Source: course PDF, pp. 10

Why Serial Printing Can Stall

GENERATED	TRANSMITTED	RESULT
12,000 B/s	**11,520 B/s**	**Buffer pressure**
120-byte message at 100 Hz	115200 baud with 8-N-1 framing	When writes cannot queue, they can wait for UART progress.

A 120-byte frame takes about 10.42 ms on the wire.

Source: course PDF, pp. 10

Move Telemetry to a Slower Task

CONTROL	TELEMETRY	UART LOAD
100 Hz	**10 Hz**	**10.4%**
Update the PWM without verbose serial output.	120 bytes × 10 Hz = 1,200 B/s	1,200 / 11,520 Leaves substantial wire capacity.

Separate control timing from communication timing.

Source: course PDF, pp. 10–11

Format Data for the Serial Plotter

```
Serial.printf("PWM_Duty: %u, Theoretical_V: %.3f\n",
              unsigned(duty), duty * 3.3f / 255.0f);
```

Use label: value pairs, commas between metrics, and a newline.

Source: course PDF, pp. 11

Check UART with a Multimeter

Power OFF	Check TX-to-RX continuity. Check that TX is not shorted to GND.
Idle, power ON	A configured 3.3 V UART TX line is normally HIGH when idle.
Active traffic	The meter shows an average voltage. Use a logic analyzer to decode bytes.

A multimeter checks electrical state, not baud rate or packet contents.

Source: course PDF, pp. 11–12

Check the 3.3 V Supply

Probe	Measure between 3V3 and GND while the scheduler runs.
Observe	Look for a steady supply near 3.3 V during computation and transmission.
If it resets	Inspect USB power, connections, and voltage drops under load.

A normal meter reading can miss short supply dips.

Source: course PDF, pp. 12

Lab Connections

GPIO 18	PWM output → resistor → LED → GND
GPIO 4	Heartbeat output → resistor → LED → GND
USB + GND	USB powers the board and carries logs. Share a ground reference for measurements.

Use separate current-limiting resistors and verify LED polarity.

Source: course PDF, pp. 13–17

Before Connecting USB

Ground path	Confirm continuity along the intended common ground rail.
Power rails	Check for a persistent low-resistance short between 3V3 and GND.
Wiring	Check GPIO 18, GPIO 4, resistor paths, and LED polarity before power-up.

Use continuity and resistance modes only with power disconnected.

Source: course PDF, pp. 13

One Loop, Three Task Rates

MOTION	TELEMETRY	HEARTBEAT
100 Hz	**10 Hz**	**1 Hz task**
Every 10 ms Update the PWM duty ramp.	Every 100 ms Print state and timing metrics.	Every 1 second Toggle the status LED on GPIO 4.

Check each timer independently; keep every task short.

Source: course PDF, pp. 14–19

Firmware: Configure the Hardware

```
Serial.begin(115200);
pinMode(4, OUTPUT);

if (!ledcAttachChannel(18, 20000, 8, 0)) {
  while (true) delay(1000);
}

ledcWrite(18, 0);
```

GPIO 18: 20 kHz PWM, 8-bit duty. GPIO 4: heartbeat LED.

Firmware: Schedule Three Tasks

```
if (uint32_t(micros() - lastMotion) >= 10000) {
  lastMotion += 10000;
  task_motion_control();
}

if (uint32_t(millis() - lastTelemetry) >= 100) {
  lastTelemetry += 100;
  task_telemetry();
}

if (uint32_t(millis() - lastHeartbeat) >= 1000) {
  lastHeartbeat += 1000;
  task_heartbeat();
}
```

Inside loop(): motion 100 Hz · telemetry 10 Hz · heartbeat 1 Hz.

Firmware: Apply, Reverse, and Step

```
void task_motion_control() {
  ledcWrite(18, duty);

  if (duty == 255) direction = -1;
  else if (duty == 0) direction = 1;

  duty += direction;
}
```

One duty step every 10 ms: 0 → 255 → 0 in about 5.1 seconds.

Firmware: Telemetry and Heartbeat

```
void task_telemetry() {
  Serial.printf("PWM_Duty: %u\n",
                unsigned(ledcRead(18)));
}

void task_heartbeat() {
  ledState = !ledState;
  digitalWrite(4, ledState);
}
```

Print at 10 Hz; toggle GPIO 4 once per second.

Verify Data

Upload + observe	Use Serial Monitor at 115200 baud. Expect about 10 telemetry lines per second.
Plot + measure	Plot the duty ramp; measure GPIO 18. Its average voltage rises and falls.
Compare	GPIO 4 changes state every second. Check a steady 3.3 V supply and task timing.

Compare the physical outputs with the firmware’s predicted behavior.

Source: course PDF, pp. 19

Debugging

Unreadable serial	Match the monitor to 115200 baud; confirm the correct USB serial port.
Erratic heartbeat	Remove long waits from loop tasks; keep verbose logging at 10 Hz.
Timing spikes	Measure task execution time; move formatting out of the 100 Hz task.

Check communication settings, blocking calls, and task execution time.

Source: course PDF, pp. 20

References

Making Embedded Systems: Design Patterns for Great Software
Elecia White

The Art of Electronics: Student Manual
Thomas C. Hayes and Paul Horowitz

Computer Systems: A Programmer's Perspective
Randal E. Bryant and David R. O'Hallaron

Embedded Software and Deterministic Multitasking
DCLab / course notes

ESP32 Arduino Core Documentation: Serial (UART)
Espressif Systems

ESP32 Programming Guide: General Purpose Timer (GPTimer)
Espressif Systems

Arduino Core for the ESP32
Espressif Systems

ESP32 Arduino Core Documentation: LED Control (LEDC)
Espressif Systems

Thank you!

Questions & discussion



TIMING SCHEDULING UART